

Chicken Processing System - Data Schema

Overview

This document defines the complete data schema for the Chicken Processing System, including Firestore collections, document structures, relationships, and validation rules.

Database: Firestore

Collections Structure

```
firestore-root/
├── users/                # User accounts
│   ├── {userId}
│   └── ...
├── products/            # Product catalog
│   ├── {productId}
│   └── ...
├── orders/              # Customer orders
│   ├── {orderId}
│   └── ...
├── batches/             # Production batches
│   ├── {batchId}
│   └── ...
├── inventory/           # Inventory tracking
│   ├── {inventoryId}
│   └── ...
├── compliance/          # Compliance checks
│   ├── {checkId}
│   └── ...
└── analytics/           # System analytics
    ├── {analyticId}
    └── ...
```

Collection Schemas

1. Users Collection

Path: `/users/{userId}`

Document Structure:

```
{
  // Entity fields
  id: string,           // Auto-generated document ID
  createdAt: Timestamp, // Creation timestamp
}
```

```

updatedAt: Timestamp,          // Last update timestamp

// User identification
email: string,                 // Unique email address
firebaseUid: string,          // Firebase Auth UID
userType: string,              // 'consumer' | 'restaurant' | 'retailer' |
'distributor' | 'farm_gate' | 'institution'
role: string,                  // 'farm_manager' | 'poultry_attendant' |
'processing_staff' | 'sales_assistant' | 'customer' | 'admin'

// Profile information
profile: {
  businessName?: string,       // Optional business name
  businessRegistrationNumber?: string,
  taxId?: string,

  address: {
    street: string,
    city: string,
    state: string,
    country: string,
    postalCode: string
  },

  contact: {
    phone: string,
    email: string,
    secondaryPhone?: string,
    website?: string
  },

  certifications?: {
    foodSafety?: string,       // Certification ID
    halal?: boolean,
    organic?: boolean,
    other?: string[]
  },

  preferences?: {
    emailNotifications: boolean,
    smsNotifications: boolean,
    marketingEmails: boolean,
    language: string,           // e.g., 'en', 'af', 'zu'
    timezone: string            // e.g., 'Africa/Johannesburg'
  }
},

// Account status
isActive: boolean,
accountStatus: string,         // 'pending' | 'approved' | 'suspended' |
'rejected'
approvedAt?: Timestamp,
rejectedAt?: Timestamp,

```

```

rejectedReason?: string,

// Security
lastLoginAt?: Timestamp,
loginAttempts: number,
lockedUntil?: Timestamp,

// Role-specific fields
department?: string,           // For farm staff
loyaltyPoints?: number         // For customers
}

```

Indexes:

- `email` (unique)
- `firebaseUid` (unique)
- `userType`
- `accountStatus`
- `createdAt` (descending)

Validation Rules:

- `email`: Required, valid email format
- `userType`: Must be one of defined enum values
- `role`: Must be one of defined enum values
- `accountStatus`: Must be one of defined enum values

2. Products Collection

Path: `/products/{productId}`

Document Structure:

```

{
  // Entity fields
  id: string,
  createdAt: Timestamp,
  updatedAt: Timestamp,

  // Product information
  name: string,           // Product name
  description: string,    // Product description
  type: string,           // 'whole_chicken' | 'breast' | 'thighs' |
                          // 'wings' | 'drumsticks' | 'gizzards' | 'feet' | 'eggs'

  // Pricing
  price: {
    amount: number,       // Price amount
    currency: string      // Currency code (default: 'ZAR')
  },
}

```

```

// Inventory
stockQuantity: number,           // Current stock level
minStockLevel: number,          // Minimum stock before reorder
maxStockLevel: number,          // Maximum stock capacity

// Product details
sku?: string,                   // Stock Keeping Unit
barcode?: string,               // Barcode/EAN
weight?: number,                // Weight in kg
unit?: string,                  // Unit of measurement
shelfLifeDays?: number,         // Days until expiration

// Type-specific fields
eggSize?: string,               // For eggs: 'small' | 'medium' | 'large' |
'extra_large'
eggGrade?: string,              // For eggs: 'A' | 'B' | 'C'
cutType?: string,               // For chicken parts: 'boneless' | 'bone_in'
packaging?: string,             // Packaging type

// Status
isActive: boolean,
isFeatured: boolean,

// Categories
categories: string[],           // e.g., ['fresh', 'premium', 'organic']

// Images
images: string[],               // Array of image URLs

// Metadata
tags: string[],
notes?: string
}

```

Indexes:

- `type`
- `isActive`
- `stockQuantity` (for low stock alerts)
- `createdAt` (descending)
- `price.amount` (for price filtering)

3. Orders Collection

Path: `/orders/{orderId}`

Document Structure:

```

{
  // Entity fields
  id: string,
  createdAt: Timestamp,
  updatedAt: Timestamp,

  // Order identification
  orderNumber: string,           // Generated order number (e.g., ORD-20250119-001)
  userId: string,               // Reference to users/{userId}

  // Order items
  items: Array<{
    productId: string,           // Reference to products/{productId}
    productName: string,        // Snapshot of product name at time of order
    quantity: number,
    unitPrice: { amount: number, currency: string },
    totalPrice: { amount: number, currency: string },
    discountApplied?: { amount: number, currency: string },
    batchNumber?: string        // Reference to production batch
  }>,

  // Pricing summary
  subtotal: { amount: number, currency: string },
  discountAmount: { amount: number, currency: string },
  shippingCost: { amount: number, currency: string },
  taxAmount: { amount: number, currency: string },
  totalAmount: { amount: number, currency: string },

  // Order status
  status: string,               // 'pending' | 'confirmed' | 'processing' | 'shipped' | 'delivered' | 'cancelled'
  paymentStatus: string,        // 'pending' | 'paid' | 'failed' | 'refunded'

  // Payment details
  paymentDetails: {
    method: string,             // 'credit_card' | 'debit_card' | 'bank_transfer' | 'mobile_money' | 'cash_on_delivery'
    transactionId?: string,
    paymentIntentId?: string,
    paidAt?: Timestamp,
    failedAt?: Timestamp,
    failedReason?: string
  },

  // Shipping details
  shippingDetails: {
    method: string,             // 'standard' | 'express' | 'pickup' | 'farm_gate' | 'local_delivery'
    carrier?: string,
    trackingNumber?: string,
    estimatedDeliveryDate?: Timestamp,
  }
}

```

```

    actualDeliveryDate?: Timestamp,
    shippingCost: { amount: number, currency: string },
    status: string // 'pending' | 'in_transit' | 'delivered' |
'failed'
  },

  // Addresses
  shippingAddress: {
    recipientName: string,
    street: string,
    city: string,
    state: string,
    country: string,
    postalCode: string,
    phone: string,
    email: string,
    instructions?: string
  },

  billingAddress: {
    companyName?: string,
    street: string,
    city: string,
    state: string,
    country: string,
    postalCode: string,
    taxId?: string
  },

  // Timestamps
  orderDate: Timestamp,
  confirmedAt?: Timestamp,
  processedAt?: Timestamp,
  shippedAt?: Timestamp,
  deliveredAt?: Timestamp,
  cancelledAt?: Timestamp,

  // Cancellation
  cancelledBy?: string, // User ID who cancelled
  cancelledReason?: string,

  // Metadata
  notes?: string,
  specialInstructions?: string,
  lastUpdatedBy: string // User ID of last updater
}

```

Indexes:

- `userId`
- `status`

- `paymentStatus`
- `orderDate` (descending)
- `totalAmount.amount` (for revenue analysis)

4. Batches Collection

Path: `/batches/{batchId}`

Document Structure:

```
{
  // Entity fields
  id: string,
  createdAt: Timestamp,
  updatedAt: Timestamp,

  // Batch identification
  batchNumber: string,           // Generated batch number (e.g., BATCH-
20250119-001)
  productionType: string,       // 'broiler_cycle' | 'egg_production' |
'hatching'

  // Production details
  startDate: Timestamp,
  expectedEndDate: Timestamp,
  actualEndDate?: Timestamp,
  status: string,               // 'planned' | 'in_progress' | 'completed' |
'cancelled'

  // Stock information
  initialStock: number,         // Initial number of chickens/eggs
  currentStock: number,         // Current available stock
  lostStock: number,            // Stock lost during production
  soldStock: number,            // Stock sold

  // Location
  location: string,             // Production location/farm
  supervisorId?: string,       // Reference to users/{userId}

  // Feed information
  feedType: string,             // 'starter' | 'grower' | 'finisher' | 'layer'
  feedConsumption: number,     // Total feed consumed (kg)

  // Health metrics
  mortalityRate: number,        // Percentage
  averageWeight: number,       // Average weight at harvest (kg)

  // Quality metrics
  qualityGrade: string,         // 'A' | 'B' | 'C'
  complianceScore: number,     // 0-100 score
}
```

```
// Products from this batch
products: Array<{
  productId: string,           // Reference to products/{productId}
  quantityProduced: number,
  quantityAvailable: number
}>,

// Documentation
documents: string[],          // Array of document URLs
images: string[],             // Array of image URLs

// Notes
notes?: string
}
```

Indexes:

- `batchNumber` (unique)
- `productionType`
- `status`
- `startDate` (descending)
- `location`

5. Inventory Collection

Path: `/inventory/{inventoryId}`

Document Structure:

```
{
  // Entity fields
  id: string,
  createdAt: Timestamp,
  updatedAt: Timestamp,

  // Product reference
  productId: string,           // Reference to products/{productId}
  batchId?: string,           // Reference to batches/{batchId}

  // Stock information
  quantity: number,           // Current quantity
  quantityChange: number,     // Change in quantity (positive for addition,
negative for deduction)
  previousQuantity: number,    // Quantity before change

  // Transaction details
  transactionType: string,     // 'purchase' | 'sale' | 'adjustment' |
'waste' | 'transfer'
  referenceId?: string,        // Reference to order/batch/etc.
  referenceType?: string,      // 'order' | 'batch' | 'manual'
}
```



```

// Location
location: string,           // Storage location
storageCondition: string,   // 'frozen' | 'refrigerated' | 'room_temp'

// Expiry
expiryDate?: Timestamp,
daysUntilExpiry?: number,

// Pricing (for cost calculation)
unitCost: { amount: number, currency: string },
totalCost: { amount: number, currency: string },

// Metadata
notes?: string,
performedBy: string         // User ID who performed the transaction
}

```

Indexes:

- `productId`
- `batchId`
- `transactionType`
- `createdAt` (descending)
- `expiryDate` (for expiry alerts)

6. Compliance Collection

Path: `/compliance/{checkId}`

Document Structure:

```

{
  // Entity fields
  id: string,
  createdAt: Timestamp,
  updatedAt: Timestamp,

  // Check information
  checkType: string,           // 'food_safety' | 'quality_control' |
  'sanitation' | 'temperature'
  checkDate: Timestamp,

  // Location/Subject
  location: string,
  batchId?: string,           // Reference to batches/{batchId}
  productId?: string,         // Reference to products/{productId}

  // Performed by
  inspectorId: string,         // Reference to users/{userId}
}

```

```

// Measurements
measurements: Record<string, any>, // Key-value pairs of measurements

// Results
passed: boolean,
score: number, // 0-100 score
issues: Array<{
  description: string,
  severity: string, // 'low' | 'medium' | 'high' | 'critical'
  correctiveAction?: string,
  resolved: boolean,
  resolvedAt?: Timestamp
}>,

// Documentation
images: string[], // Array of image URLs
documents: string[], // Array of document URLs

// Notes
notes?: string
}

```

Indexes:

- `checkType`
- `checkDate` (descending)
- `passed`
- `batchId`
- `inspectorId`

7. Analytics Collection

Path: `/analytics/{analyticId}`

Document Structure:

```

{
  // Entity fields
  id: string,
  createdAt: Timestamp,
  updatedAt: Timestamp,

  // Analytics type
  analyticType: string, // 'daily_sales' | 'monthly_revenue' |
    'user_activity' | 'inventory_trends'
  period: string, // e.g., '2025-01', '2025-01-19'

  // Data
  data: Record<string, any>, // Structured analytics data
}

```

```

// Summary metrics
metrics: {
  totalOrders?: number,
  totalRevenue?: { amount: number, currency: string },
  averageOrderValue?: { amount: number, currency: string },
  newUsers?: number,
  activeUsers?: number,
  inventoryTurnover?: number,
  wastePercentage?: number
},

// Breakdowns
breakdowns: {
  ordersByStatus?: Record<string, number>,
  ordersByPaymentMethod?: Record<string, number>,
  revenueByProduct?: Array<{
    productId: string,
    productName: string,
    revenue: { amount: number, currency: string },
    quantity: number
  }>,
  usersByType?: Record<string, number>
},

// Trends
trends: Array<{
  date: string,
  value: number,
  metric: string
}>,

// Generated at
generatedAt: Timestamp
}

```

Indexes:

- `analyticType`
- `period`
- `createdAt` (descending)

Relationships

1. User Relationships

- **One-to-Many:** User → Orders (user can have multiple orders)
- **One-to-Many:** User → Compliance Checks (inspector can perform multiple checks)

2. Product Relationships

- **One-to-Many:** Product → Order Items (product can be in multiple orders)
- **One-to-Many:** Product → Inventory Transactions (product can have multiple inventory transactions)
- **Many-to-One:** Product ← Batch (product can come from a batch)

3. Order Relationships

- **Many-to-One:** Order → User (order belongs to a user)
- **One-to-Many:** Order → Order Items (order contains multiple items)

4. Batch Relationships

- **One-to-Many:** Batch → Products (batch produces multiple products)
- **One-to-Many:** Batch → Inventory Transactions (batch affects multiple inventory items)
- **One-to-Many:** Batch → Compliance Checks (batch can have multiple checks)

5. Inventory Relationships

- **Many-to-One:** Inventory → Product (inventory item is for a product)
- **Many-to-One:** Inventory → Batch (inventory item may come from a batch)

Data Validation Rules

Firestore Security Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Users collection
    match /users/{userId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null &&
        (request.auth.uid == userId ||

get(/databases/{database}/documents/users/{request.auth.uid}).data.role ==
'admin');
    }

    // Products collection
    match /products/{productId} {
      allow read: if true; // Public read
      allow write: if request.auth != null &&

get(/databases/{database}/documents/users/{request.auth.uid}).data.role in
['admin', 'farm_manager', 'sales_assistant'];
    }

    // Orders collection
    match /orders/{orderId} {
      allow read: if request.auth != null;
      allow create: if request.auth != null;
```

```

    allow update: if request.auth != null &&
                      (request.auth.uid == resource.data.userId ||

get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role in
['admin', 'sales_assistant']);
    allow delete: if
get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role ==
'admin';
  }

  // Batches collection
  match /batches/{batchId} {
    allow read: if request.auth != null;
    allow write: if request.auth != null &&

get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role in
['admin', 'farm_manager'];
  }

  // Inventory collection
  match /inventory/{inventoryId} {
    allow read: if request.auth != null;
    allow write: if request.auth != null &&

get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role in
['admin', 'farm_manager', 'processing_staff'];
  }

  // Compliance collection
  match /compliance/{checkId} {
    allow read: if request.auth != null;
    allow write: if request.auth != null &&

get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role in
['admin', 'farm_manager', 'poultry_attendant'];
  }

  // Analytics collection
  match /analytics/{analyticId} {
    allow read: if request.auth != null;
    allow write: if
get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role ==
'admin';
  }
}
}
}

```

Data Integrity Rules

1. Referential Integrity:

- When a user is deleted, their orders should be archived (not deleted)
- When a product is deleted, it should be soft-deleted (isActive = false)
- Batch references must point to valid batch documents
- Product references must point to valid product documents

2. Business Rules:

- Order total must equal sum of item totals minus discounts plus shipping and tax
- Stock quantity cannot go below zero
- Order status transitions must follow valid sequence
- User account status must follow valid transitions

3. Validation Constraints:

- Email addresses must be unique
- Order numbers must be unique
- Batch numbers must be unique
- SKU codes must be unique (if provided)
- Prices must be positive numbers
- Quantities must be positive integers

Index Requirements

Required Composite Indexes:

1. Orders Collection:

- `userId + orderDate` (descending)
- `status + orderDate` (descending)
- `paymentStatus + orderDate` (descending)

2. Products Collection:

- `type + isActive + createdAt` (descending)
- `stockQuantity + isActive`

3. Inventory Collection:

- `productId + createdAt` (descending)
- `batchId + transactionType`

4. Batches Collection:

- `productionType + status + startDate` (descending)
- `location + startDate` (descending)

Data Migration Strategy

Phase 1: Initial Setup

1. Create Firestore database with collections

2. Set up security rules
3. Create required indexes
4. Seed initial data (admin users, product categories)

Phase 2: Data Migration

1. Migrate existing user data to new schema
2. Migrate product catalog
3. Migrate historical orders (if any)
4. Set up batch tracking

Phase 3: Validation

1. Run data validation scripts
2. Test security rules
3. Verify referential integrity
4. Performance testing

Backup and Recovery

Daily Backups

- Automated Firestore exports to Cloud Storage
- Retention: 30 days of daily backups
- Monthly archives retained for 1 year

Recovery Procedures

1. **Point-in-time recovery:** Restore from Firestore backup
2. **Partial recovery:** Restore specific collections
3. **Data corruption:** Use transaction logs for recovery

Monitoring and Alerts

Key Metrics to Monitor

1. **Database Size:** Collection document counts
2. **Read/Write Operations:** Operation counts and latency
3. **Security Rule Denials:** Failed access attempts
4. **Index Performance:** Index usage and efficiency

Alert Conditions

- Database size exceeds 90% of quota
- Read/write latency > 1000ms
- Security rule denials spike
- Index creation failures

Conclusion

This data schema provides a comprehensive foundation for the Chicken Processing System, with:

- Clear collection structures and relationships
- Proper validation and security rules
- Scalable indexing strategy
- Backup and recovery procedures
- Monitoring and alerting guidelines

The schema is designed to support the business requirements while maintaining data integrity, security, and performance.